Quantum Advantage for QUBO

Solving QUBO problems with Grover Adaptive Search

Ahmed Raza · Student No. 2026210028 · 9 November 2026

1. Problem

A **QUBO** (Quadratic Unconstrained Binary Optimisation) problem asks for the string of n bits that makes a quadratic cost as small as possible:

$$\min_{x \in \{0,1\}^n} f(x) = x^\top Q x. \tag{1}$$

Many real tasks, such as scheduling, routing and portfolio selection, can be written in this form. The difficulty is that there are $N = 2^n$ possible answers, and in general no fast way to find the best one. Adding one variable doubles the search space.

As a test problem, **Max-Cut** is used: split the nodes of a graph into two groups so that as many edges as possible connect the two groups. If x_i tells which group node i is in, the number of cut edges is

$$\text{cut}(x) = \sum_{(i,j) \in E} (x_i + x_j - 2x_i x_j) = -x^\top Q x, \qquad Q_{ii} = -\deg(i), \;\; Q_{ij} = 1 \text{ if } (i,j) \in E, \tag{2}$$

so maximising the cut is the same as minimising the QUBO cost (1) [5]. Random graphs in which every node has three neighbours are used, with n = 6 to 24 variables.

The question. A quantum computer has an *advantage* when it needs less work than a classical computer, and the gap grows as the problem gets bigger. Here, work is counted in **queries**: one query is one evaluation of the cost f(x). The goal is to check whether a quantum algorithm needs fewer queries than classical search to find the best answer of a QUBO.

2. Method: Grover Adaptive Search

Grover search [4] can find a "marked" item among N items with about √N steps instead of N. **Grover Adaptive Search** (GAS) [1], [2] uses this idea to find the minimum of a QUBO:

- Start with a random answer x and set a threshold y = f(x).
- Mark every answer that is better than the threshold, i.e. f(x) < y. On a quantum computer this is done by an *oracle* that flips the sign of the marked states.
- Start from an equal superposition of all N answers and apply k Grover steps (oracle + "diffusion"). Each step moves probability from the unmarked answers to the marked ones. Then measure.
- If the measured answer is better, it becomes the new threshold. Repeat until no better answer is found.

If t answers are marked, the chance of measuring one of them after k steps is

$$P(k) = \sin^2((2k+1)\theta), \qquad \sin^2\theta = \tfrac{t}{N}, \qquad \text{best } k \approx \tfrac{\pi}{4}\sqrt{N/t}. \tag{3}$$

A classical random guess finds a marked answer with chance only t/N. Since t is not known in advance, k is chosen at random from a growing range [3].

Comparison. GAS is compared with two classical methods that use the same access to f: *exhaustive search* (check all N answers) and *random search* (pick random answers and keep the best). Each method runs until it first finds the true optimum, and every query is counted: one oracle call or one classical evaluation. Three graphs per size and 40 runs per graph are used. The quantum circuit is simulated exactly in Python: a full state vector up to 18 qubits, and the identical formula (3) for 20–24 qubits.

3. Results

Fig. 1 shows one 12-variable problem with 2 optimal answers among 4,096. One random guess finds an optimum with chance 0.0005. After 35 Grover steps the chance is almost 1. The simulation (dots) matches the formula (3) exactly.

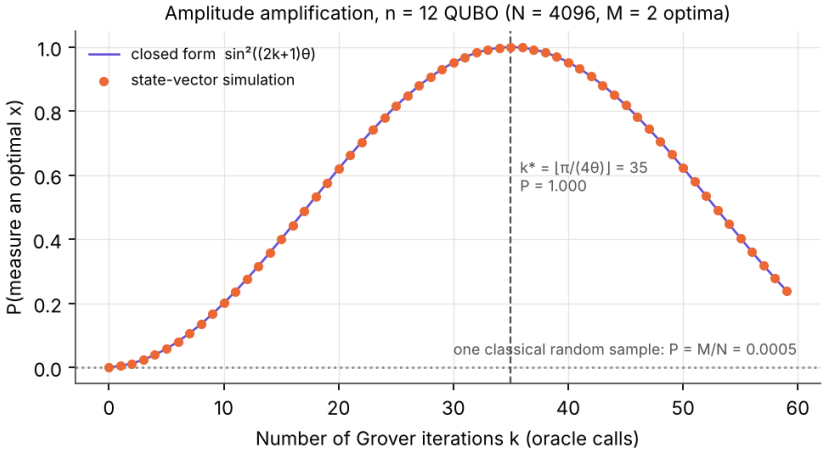


Fig. 1. Chance of measuring an optimal answer after k Grover steps (12 variables, N = 4,096, M = 2 optima).

Table I and Fig. 2 show how many queries each method needs to find the optimum. GAS always needs far fewer, and the gap grows quickly with size.

Variables n	GAS	Random search	Exhaustive search	GAS speed-up vs random	vs exhaustive
8	20	86	256	4×	13×
12	105	2,349	4,096	22×	39×
16	300	23,414	65,536	78×	218×
20	958	269,759	1,048,576	281×	1,094×
24	4,088	4,112,993	16,777,216	1,006×	4,104×

Table I. Average number of queries to find the optimum (3 graphs × 40 runs per size).

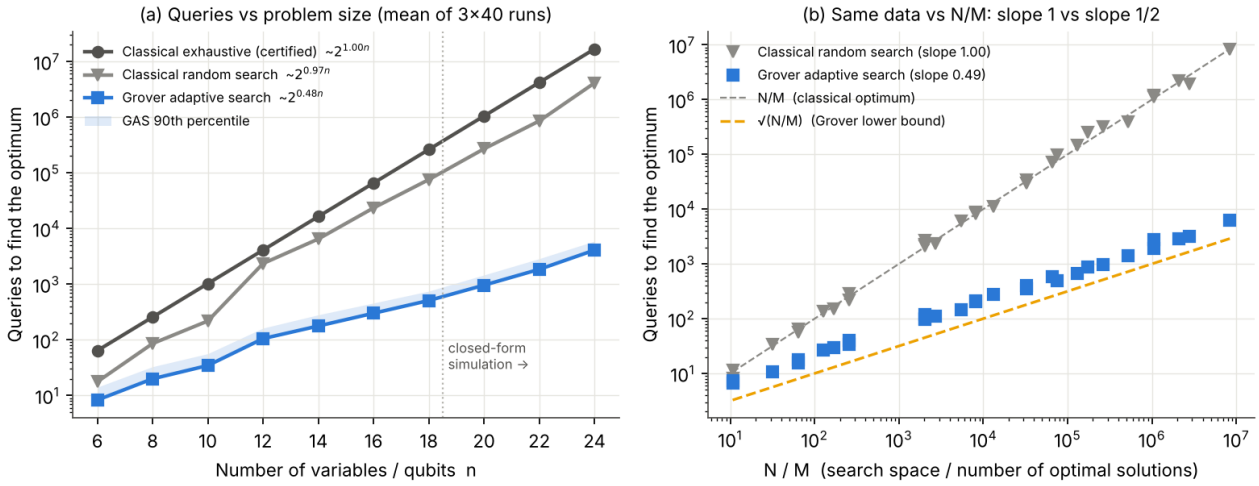


Fig. 2. Queries to find the optimum, log scale. (a) Against problem size. (b) Against N/M (search space divided by number of optima): classical search follows N/M, GAS follows √(N/M).

Fig. 2b shows the key result. When the queries are plotted against N/M (M = number of optimal answers), classical search grows with slope 1.00 and GAS with slope 0.49. In other words, classical search needs about N/M queries and GAS about √(N/M), exactly as theory predicts. The speed-up is about 22× at 12 variables and about 1,006× at 24 variables (Fig. 3), and it keeps growing.

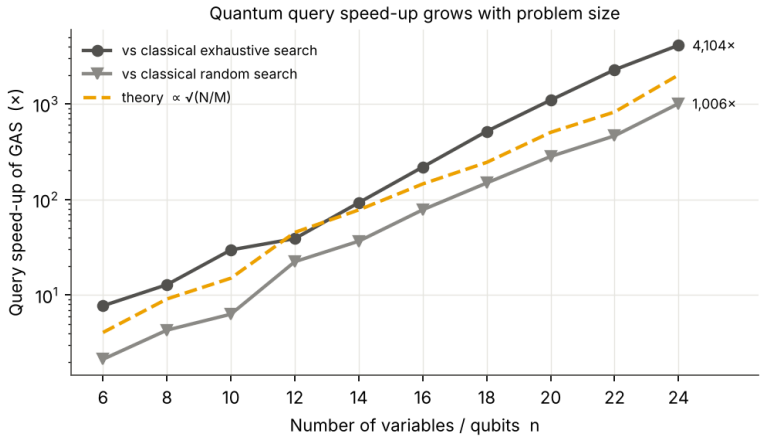


Fig. 3. How many times fewer queries GAS needs, compared with the theoretical √(N/M) growth.

4. Conclusion

We checked whether a quantum computer can solve QUBO problems with less work than a classical computer. Max-Cut was written as a QUBO and solved with Grover Adaptive Search, and the results were compared with exhaustive and random classical search on problems with 6 to 24 variables.

The answer is **yes**. GAS needed about √(N/M) queries, while classical search needed about N/M. The measured slopes (0.49 vs 1.00) match the theory, and the saving grew from a few times on small problems to about 1,006× fewer queries than random search, and 4,104× fewer than checking every answer, at 24 variables. This is a true quantum advantage, and it is also **proven**: no classical method can beat about N/M queries without extra knowledge of the problem, and no quantum method can beat about √(N/M) [6], so Grover's quadratic speed-up is the best possible [1].

The result also has clear limits. The advantage is *quadratic*, not exponential, so the problem stays hard for both computers. It is measured against *blind* search; smart classical heuristics such as simulated annealing use the structure of the problem and can be much faster on easy instances. Finally, each Grover step needs a deep circuit that computes f(x), so a real speed-up in seconds needs large, error-corrected quantum computers that do not exist yet [7].

In summary, Grover Adaptive Search gives QUBO a real and provable quantum advantage in the number of queries, and this experiment confirms it. Turning it into a practical advantage will need better quantum hardware, and probably combining Grover's speed-up with good classical heuristics.

References

[1] C. Dürr, P. Høyer, “A quantum algorithm for finding the minimum,” arXiv:quant-ph/9607014, 1996.

[2] A. Gilliam, S. Woerner, C. Gonciulea, “Grover adaptive search for constrained polynomial binary optimization,” *Quantum*, vol. 5, 428, 2021.

[3] M. Boyer, G. Brassard, P. Høyer, A. Tapp, “Tight bounds on quantum searching,” *Fortschr. Phys.*, vol. 46, pp. 493–505, 1998.

[4] L. K. Grover, “A fast quantum mechanical algorithm for database search,” *Proc. 28th ACM STOC*, pp. 212–219, 1996.

[5] A. Lucas, “Ising formulations of many NP problems,” *Front. Phys.*, vol. 2, 5, 2014.

[6] C. H. Bennett, E. Bernstein, G. Brassard, U. Vazirani, “Strengths and weaknesses of quantum computing,” *SIAM J. Comput.*, vol. 26, no. 5, pp. 1510–1523, 1997.

[7] R. Babbush et al., “Focus beyond quadratic speedups for error-corrected quantum advantage,” *PRX Quantum*, vol. 2, 010103, 2021.